

Genetic Algorithm Assignment – Maze Navigation Problem

Şeyma COŞTUR
2540014021

Abstract—This report was prepared as a term paper for the Evolutionary Computing course. The aim of the study is to find the optimum path for an agent to reach the target point (G) from the starting point (S) in two different maze environments with dimensions (10×10 and 20×20) using the Genetic Algorithm (GA) method. Each chromosome encodes a fixed-length sequence of directional moves (U, D, L, R). A custom fitness function rewards goal-reaching and path efficiency while penalizing wall hits, out-of-bounds moves, and repeated positions. Experiments show that both mazes were successfully solved; however, the 20×20 maze required significantly more generations and runtime, demonstrating how problem size directly affects GA performance.

I. INTRODUCTION

Maze navigation is a common combinatorial search problem and is frequently used to evaluate evolutionary algorithms. In this problem, a 2D grid is defined with walls, a start point (S), and a goal point (G). The objective is to determine a sequence of moves that enables an agent to reach the goal without colliding with walls or moving outside the grid.

Genetic Algorithms are well-suited to this problem, as they explore large search spaces using a population-based approach. Through selection, crossover, and mutation, candidate solutions are iteratively refined. In this study, a standard Genetic Algorithm is implemented and tested on two different maze sizes to analyze the effects of problem complexity on convergence speed, solution quality, and parameter sensitivity.

II. PROBLEM DESCRIPTION

The maze is represented as a 2D grid where each cell can be a free space (0), a wall (1), a start position (S), or a goal position (G). The agent starts at S and tries to reach G by following a sequence of moves. Each move is one of four directions: Up (U), Down (D), Left (L), or Right (R). If a move would cause the agent to hit a wall or go outside the grid, it is considered invalid, and the agent stays in the same position.

III. GENETIC ALGORITHM DESIGN

A. Chromosome Representation

Each individual in the population is a fixed-length list of moves drawn from {U, D, L, R}. The chromosome length determines the maximum number of steps the agent can take. A longer chromosome allows the agent to explore deeper paths but also increases the search space. For Maze 1 a baseline length of 40 was used, and for Maze 2 a baseline length of 120 was used, reflecting the larger navigation distance required.

However, three different values were tested for each parameter in both mazes.

B. Population Initialization

The initial population is generated randomly. Each gene in every chromosome is independently assigned a random direction with equal probability. This helps explore a wide range of possible solutions.

C. Fitness Function

The fitness function is the core of the GA design. It evaluates each chromosome by simulating its execution on the maze and assigning a score based on four criteria. If the agent reaches the goal, the score starts at 1000 and is reduced by penalties. I was inspired by [1] when formulating the penalty equation.

$$f = 1000 - 0.5 \text{ steps} - p \text{ wall_hits} - p \text{ oob} - 0.5p \text{ loops} \quad (1)$$

If the goal is not reached, the score is negative and based on Manhattan distance to the goal:

$$f = -2 \text{ dist} - p \text{ wall_hits} - p \text{ oob} - 0.5p \text{ loops} \quad (2)$$

where p is the penalty coefficient and dist is the Manhattan distance from the agent's final position to the goal. A penalty is applied at each step; the penalty increases if the path is longer. Penalties for hitting the wall and out-of-bounds deduct points from your fitness rating, however, these actions are considered invalid and do not affect algorithm progress. The loop penalty discourages revisiting the same cell repeatedly.

Inspired by [1], a penalty-based approach is used in the fitness function. However, in earlier implementations, penalty values tended to grow excessively, which led to very large negative fitness values in later generations. To address this issue, the penalty formulation was improved by introducing a bounded design.

Two penalty scales are used in this study. A fixed penalty ($p = 10$) is applied for fitness history logging and for comparing individuals, ensuring consistent and comparable values across generations. In addition, an escalating selection penalty (starting at 10, doubling every 30 generations, and capped at 160) is used only during tournament selection to gradually increase selection pressure over time. This improved design prevents the fitness values from collapsing to very large negative values in later generations.

D. Selection

Tournament selection is used. In each tournament, a random subset of individuals is selected and the one with the highest fitness wins. The tournament size controls selection pressure: a larger tournament produces stronger pressure toward high-fitness individuals but reduces diversity. A tournament size of 4 was used as the baseline for Maze 1, and 5 for Maze 2. The tournament size parameter is set differently for each maze. For Maze 1, the tested values are 2, 4, and 7, while for Maze 2, the values are 2, 5, and 8.

E. Crossover

Single-point crossover is applied with a configurable probability (crossover rate). A random cut point is chosen, and the child inherits the left segment from parent 1 and the right segment from parent 2. If crossover is not applied, the child is a copy of parent 1. This operator allows the GA to combine partial solutions from two parents.

F. Mutation

Each gene in the child chromosome is independently replaced with a random direction with probability equal to the mutation rate. Mutation introduces new genetic material and prevents premature convergence. Too low a mutation rate causes the population to stagnate, while too high a rate effectively turns the search into random sampling. This trade-off was clearly observed in the Maze 2 experiments.

G. Replacement Strategy and Elitism

Generational replacement is used, meaning the entire population is replaced in each generation. Two elitism slots are reserved. The individual with the highest fitness in the current generation is always carried over to the next generation. In addition, once an individual reaches the goal, that solved individual is also preserved in all subsequent generations. This ensures that a found solution is not lost. It also fixes an issue in earlier versions of the code where a previously found solution could sometimes be lost due to fluctuations in fitness values.

IV. PARAMETER TUNING

Parameter tuning was conducted using a *ceteris paribus* approach: one parameter is varied across three values while all others are held fixed at the baseline. This isolates the effect of each individual parameter. The results for both mazes are shown in Tables I and II.

TABLE I
PARAMETER TUNING RESULTS — MAZE 1 (10×10)

Parameter	Value	Result	Gen	Fitness	Time (s)
pop_size	30	✓	12	913.0	0.51
	80	✓	56	913.0	1.65
	150	✓	8	963.0	3.08
chrom_length	20	×	—	-32.0	1.22
	40	✓	8	931.0	0.85
	60	✓	2	936.0	1.00
generations	100	✓	5	942.0	0.27
	300	✓	8	920.0	0.86
	500	✓	6	963.0	1.31
mutation_rate	0.01	✓	6	917.0	0.79
	0.05	✓	10	942.0	0.82
	0.10	✓	6	969.0	0.84
crossover_rate	0.5	✓	22	964.0	0.78
	0.8	✓	3	918.0	0.87
	0.95	✓	10	941.0	0.85
tournament_size	2	✓	10	970.0	0.80
	4	✓	1	951.0	1.35
	7	✓	6	924.0	0.91

TABLE II
PARAMETER TUNING RESULTS — MAZE 2 (20×20)

Parameter	Value	Result	Gen	Fitness	Time (s)
pop_size	150	✓	65	645.0	3.88
	200	✓	41	755.0	9.93
	400	✓	14	677.0	21.45
chrom_length	70	×	—	-205.0	8.26
	120	✓	29	792.0	9.66
	180	✓	10	787.0	13.87
generations	200	✓	16	666.0	4.04
	500	✓	11	669.0	9.82
	800	✓	23	722.0	15.72
mutation_rate	0.01	×	—	-416.0	15.78
	0.07	✓	29	696.0	9.39
	0.15	✓	20	466.0	9.95
crossover_rate	0.5	✓	55	786.0	10.11
	0.85	✓	19	772.0	9.87
	0.95	✓	60	764.0	10.11
tournament_size	2	×	—	-539.0	13.15
	5	✓	69	726.0	10.18
	8	✓	48	747.0	10.06

V. RESULTS AND DISCUSSION

A. Final Run Results

The final run was performed using the best configuration identified from the tuning phase. For Maze 1 the best configuration was pop=80, chrom=40, gen=300, mutation=0.05, crossover=0.8, tournament=2. For Maze 2 the best configuration was pop=200, chrom=120, gen=500, mutation=0.07, crossover=0.85, tournament=5.

TABLE III
COMPARISON OF MAZE 1 AND MAZE 2 PERFORMANCE METRICS

Metric	Maze 1 (10×10)	Maze 2 (20×20)
Solved?	True	True
First solution generation	8	60
Stable solution count	292	440
Best fitness	952.0	655.0
Runtime (s)	0.83	10.25

Both mazes were successfully solved. Maze 1 reached the goal at generation 8 and maintained the solution for 292 out of 300 generations. Maze 2 required significantly more generations — first solution at generation 60 — and ran for 500 generations with 440 stable solution generations.

B. Convergence Analysis

Figure 1 shows the convergence plot for both mazes. Maze 1 converges very rapidly: the fitness jumps from approximately -200 at generation 0 to above 800 within the first 10 generations, then gradually improves toward 952. This reflects the relatively simple search space of the 10×10 grid.

Maze 2 shows a very different convergence pattern. The initial fitness is around -750, reflecting the large Manhattan distance to the goal. Fitness remains negative and flat for approximately 50 generations before a sudden jump occurs around generation 60, which is exactly when the first solution is found. After this point the algorithm continues to refine the path, slowly improving from around 400 to 655 over the remaining 440 generations. The plateau behavior after generation 150 suggests the algorithm has converged to a local optimum and the remaining improvement is marginal.

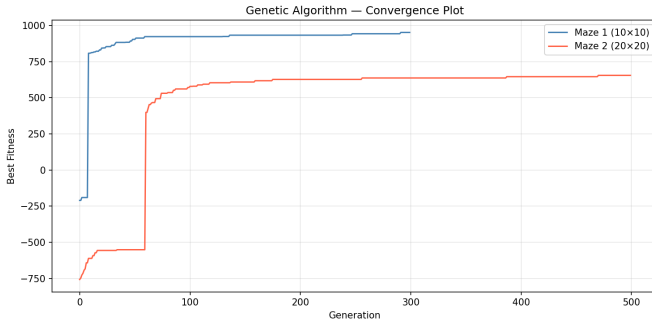


Fig. 1. Convergence Plot

C. Population Diversity Analysis

Figure 2 shows population diversity over time, measured as the number of unique individuals in each generation. For Maze 1, diversity fluctuates around 79 out of 80 individuals throughout the run. For Maze 2, diversity remains at approximately 199–200 out of 200 individuals.

These high diversity values are expected due to the chromosome lengths used (40 and 120, respectively). Since each gene can take one of four possible moves, the total number of possible chromosomes is extremely large, making duplicate individuals very unlikely. As a result, diversity alone is not a reliable indicator of convergence in this problem. The algorithm can maintain high diversity while still converging in terms of fitness.

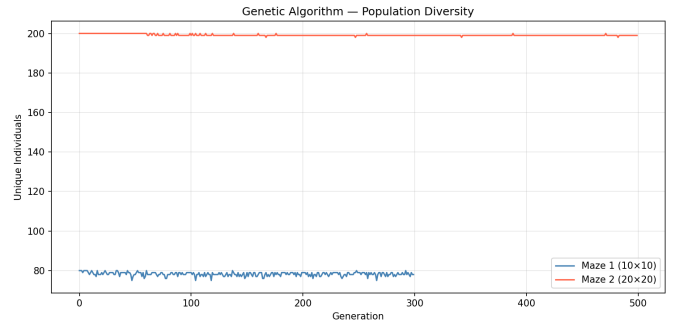


Fig. 2. Population Diversity Plot

D. Parameter Analysis

- Chromosome length was the most important parameter in both mazes. In Maze 1, a length of 20 was the only configuration that failed, as the agent stopped 6 cells away from the goal. This is expected because the shortest path in this maze requires more than 20 moves. In Maze 2, a length of 70 also failed; the agent reached position (9,19) but remained 10 cells away from the goal, showing that the chromosome was too short to represent a complete path.
- Mutation rate had a clear effect in Maze 2. A rate of 0.01 led to complete failure, most likely due to premature convergence, where the population lost diversity too early. A rate of 0.15 produced a valid solution but with a lower fitness (466) compared to 696 at 0.07, since high mutation disrupts useful partial solutions. In Maze 1, all mutation rates worked, which suggests the simpler maze is less sensitive to this parameter.
- Tournament size also had a strong effect in Maze 2. A size of 2 resulted in failure with a fitness of -539, indicating that the selection pressure was too weak and good solutions were not sufficiently exploited. Sizes of 5 and 8 both succeeded, with size 8 giving slightly better fitness.
- Population size showed a trade-off between performance and runtime. In Maze 2, a population of 80 produced a valid solution but with lower fitness (645) and higher variability. A population of 200 achieved better fitness (755). A population of 400 found a solution earlier (generation 14), but with lower final fitness (677), likely because the higher computational cost reduced the number of effective improvements within the same time limit.
- Crossover rate and number of generations had relatively small effects in both mazes, suggesting that once other parameters are well tuned, these factors have less influence on performance.

VI. COMPARISON BETWEEN MAZE SIZES

Which maze was harder?

Maze 2 was clearly more difficult. Both mazes were eventually solved, but Maze 1 reached a solution very quickly, around generation 8. Maze 2, on the other hand, took much

longer, around generation 60. The final fitness values (952 for Maze 1 and 655 for Maze 2) also show that Maze 2 produced less efficient solutions with more unnecessary moves.

Did the number of generations increase?

Yes, quite a lot. In Maze 1, a solution was usually found within the first 10 generations. For Maze 2, it often took between 10 and 70 generations, and in some cases, the algorithm failed to find a solution at all.

Did runtime increase?

Yes, significantly. Maze 1 finished in about 0.83 seconds, while Maze 2 took around 10.25 seconds. This is roughly 12 times longer, mainly because of the larger population size, longer chromosomes, and more generations.

Did parameters that worked for Maze 1 also work for Maze 2?

Only partly. The same parameter ranges could still be used, but they needed to be scaled up for Maze 2. For example, a chromosome length of 40 works well for Maze 1, but is not enough for Maze 2, where around 120 is needed. Similarly, a tournament size of 2 works fine for Maze 1 but is too weak for Maze 2.

Did diversity behave differently?

Not really in terms of pattern. The absolute diversity values were different (about 79 for Maze 1 and 199 for Maze 2), but this is just because of the different population sizes. In both cases, almost every individual was unique, meaning diversity stayed high throughout. This suggests that the search space is large enough that diversity naturally remains high without needing extra mechanisms.



Fig. 3. Maze1 Solution

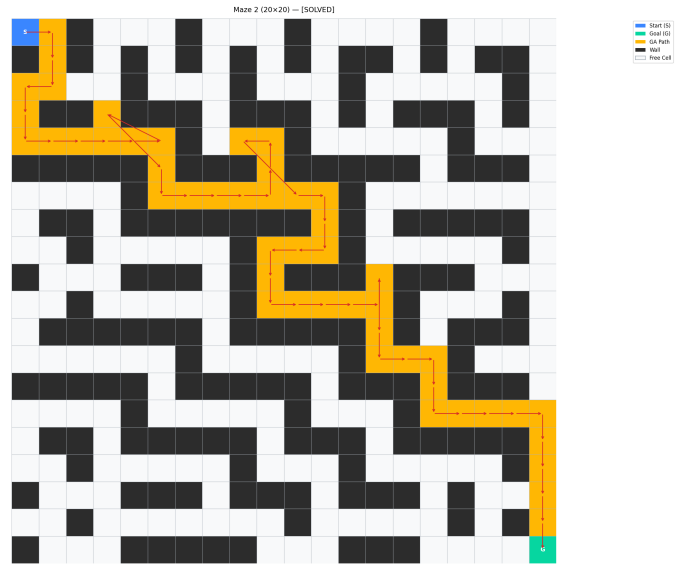


Fig. 4. Maze2 Solution

VII. CONCLUSION

This study showed that a Genetic Algorithm can successfully solve maze navigation problems of different sizes. Both the 10x10 and 20x20 mazes were solved, although with noticeably different computational costs and convergence behavior. The main design choices, including the two-scale fitness function, the use of elitism to preserve goal-reaching individuals, and the escalating selection penalty, were important for achieving stable and consistent results. The experiments also showed that chromosome length and mutation rate are the most sensitive parameters, especially in the larger maze. For future work, adaptive mutation rates, heuristic-based initialization, or variable-length chromosomes could be explored to improve efficiency and solution quality.

REFERENCES

- [1] D. J. Webb, W. M. Alobaidi, and E. Sandgren, "Maze Navigation via Genetic Optimization," *Intelligent Information Management*, vol. 10, no. 1, pp. 1–15, 2018. doi: 10.4236/iim.2018.101001. Available: <http://www.scirp.org/journal/iim>